



Toán cơ bản (Basic Math)

Nội dung

- 1. Ước số (Divisors)
 - 1.1. Ước số là gì?
 - 1.2. Cách liệt kê ước số "ngây thơ" — $\mathcal{O}(n)$
 - 1.3. Liệt kê ước số nhanh — $\mathcal{O}(\sqrt{n})$
 - 1.4. Đếm số ước
- 2. GCD (Ước chung lớn nhất) và LCM (Bội chung nhỏ nhất)
 - 2.1. GCD — Ước chung lớn nhất
 - 2.2. Thuật toán Euclid — Tìm GCD siêu nhanh
 - Chạy tay: $\gcd(48, 18)$
 - Code
 - 2.3. Ví dụ khác: $\gcd(252, 105)$
 - 2.4. LCM — Bội chung nhỏ nhất
 - Code — chú ý tránh tràn số
- 3. Số nguyên tố (Prime Numbers)
 - 3.1. Số nguyên tố là gì?
 - 3.2. Kiểm tra số nguyên tố — $\mathcal{O}(\sqrt{n})$
 - 3.3. Phiên bản tối ưu hơn
 - 3.4. Ví dụ kiểm tra
- 4. Sàng Eratosthenes — Tìm tất cả số nguyên tố đến N
 - 4.1. Ý tưởng
 - 4.2. Thuật toán
 - 4.3. Minh họa với $N = 30$
 - 4.4. Code
- 5. Số học Modular (Modular Arithmetic)
 - 5.1. Tại sao cần modular?
 - 5.2. Các tính chất cơ bản
 - 5.3. Code
- 6. Lũy thừa nhanh (Fast Exponentiation)



Tiếng Việt



- b.2. Tương
- 6.3. Chạy tay: $(3^{13} \bmod 100)$
- 6.4. Code
- 6.5. Ví dụ sử dụng

- 7. Tổng hợp: Các lỗi phổ biến
- Tóm tắt bài học

Toán học là nền tảng không thể thiếu trong lập trình thi đấu. Rất nhiều bài toán CP đòi hỏi kiến thức toán cơ bản: từ ước số, UCLN, BCNN, đến số nguyên tố, số học modular, và lũy thừa nhanh.

Đừng lo nếu bạn chưa giỏi toán — bài học này sẽ đi từng bước một, từ khái niệm đến code, với nhiều ví dụ minh họa chi tiết. Mỗi phần đều có giải thích "tại sao" chứ không chỉ "làm sao"!

1. Ước số (Divisors)

1.1. Ước số là gì?

Ước số của một số nguyên dương n là số nguyên dương d mà n chia hết cho d (tức $n \bmod d = 0$, hay nói cách khác n/d là số nguyên).

Ví dụ: Các ước số của 12 là: 1, 2, 3, 4, 6, 12.

Ta kiểm tra từng số:

D	$12 \div D$	CHIA HẾT?	LÀ ƯỚC?
1	12	Có	Ước
2	6	Có	Ước
3	4	Có	Ước
4	3	Có	Ước
5	2.4	Không	Không
6	2	Có	Ước
7	1.71...	Không	Không
...	...	...	...



Tiếng Việt



12	1	Có	Ước
----	---	----	-----

Vậy 12 có 6 ước: $\{1, 2, 3, 4, 6, 12\}$.

1.2. Cách liệt kê ước số "ngây thơ" — $O(n)$

Cách đơn giản nhất: thử tất cả số từ 1 đến n .

C++

Python

```
vector<int> divisors;
for (int i = 1; i <= n; i++) {
    if (n % i == 0) {
        divisors.push_back(i);
    }
}
```

```
divisors = []
for i in range(1, n + 1):
    if n % i == 0:
        divisors.append(i)
```

Cách này chạy được khi n nhỏ (ví dụ $n \leq 10^6$). Nhưng nếu $n = 10^{12}$ thì sao? Vòng lặp chạy 10^{12} lần — **quá chậm!**

1.3. Liệt kê ước số nhanh — $O(\sqrt{n})$

Quan sát cốt lõi: Nếu d là ước của n , thì n/d cũng là ước của n . Các ước luôn đi **thành cặp**: $(d, n/d)$.

Ví dụ với $n = 36$:



Tiếng Việt



$1 \times 36 = 36 \rightarrow \text{cặp } (1, 36)$
 $2 \times 18 = 36 \rightarrow \text{cặp } (2, 18)$
 $3 \times 12 = 36 \rightarrow \text{cặp } (3, 12)$
 $4 \times 9 = 36 \rightarrow \text{cặp } (4, 9)$
 $6 \times 6 = 36 \rightarrow \text{cặp } (6, 6) \leftarrow d = n/d = \sqrt{36}$

Nhận xét: trong mỗi cặp $(d, n/d)$, **ít nhất một số** $\leq \sqrt{n}$.

Chứng minh: Giả sử phản chứng: cả $d > \sqrt{n}$ và $n/d > \sqrt{n}$. Khi đó:

$$d \times (n/d) > \sqrt{n} \times \sqrt{n} = n$$

Nhưng $d \times (n/d) = n$ — **mâu thuẫn!**

Vì vậy, ta chỉ cần thử d từ 1 đến $\sqrt{n}$. Mỗi khi tìm được ước d , ta có luôn ước n/d .

C++

Python

```

vector<long long> divisors;
for (long long i = 1; i * i <= n; i++) {
    if (n % i == 0) {
        divisors.push_back(i);
        if (i != n / i) divisors.push_back(n / i); // tránh trùng khi i =
√n
    }
}
sort(divisors.begin(), divisors.end());

```



Tiếng Việt



```

i = 1
while i * i <= n:
    if n % i == 0:
        divisors.append(i)
        if i != n // i:
            divisors.append(n // i)
    i += 1

divisors.sort()

```

So sánh tốc độ:

N	$O(N)$ — DUYỆT HẾT	$O(\sqrt{N})$ — DUYỆT CẶP
10^6	10^6 bước	10^3 bước
10^{12}	10^{12} bước (TLE!)	10^6 bước (OK!)

1.4. Đếm số ước

Nếu chỉ cần **đếm** mà không cần liệt kê:

C++

Python

```

int count_divisors(long long n) {
    int count = 0;
    for (long long i = 1; i * i <= n; i++) {
        if (n % i == 0) {
            count++;
            if (i != n / i) count++;
        }
    }
    return count;
}

```



```
count = 0
i = 1
while i * i <= n:
    if n % i == 0:
        count += 1
        if i != n // i:
            count += 1
    i += 1
return count
```

2. GCD (Ước chung lớn nhất) và LCM (Bội chung nhỏ nhất)

2.1. GCD — Ước chung lớn nhất

$\gcd(a, b)$ là số nguyên dương **lớn nhất** mà cả a và b đều chia hết.

Ví dụ: $\gcd(12, 18) = 6$ vì:

- Ước của 12: $\{1, 2, 3, 4, 6, 12\}$
- Ước của 18: $\{1, 2, 3, 6, 9, 18\}$
- Ước chung: $\{1, 2, 3, 6\}$
- Lớn nhất: 6

2.2. Thuật toán Euclid — Tìm GCD siêu nhanh

Thay vì liệt kê tất cả ước, ta dùng **thuật toán Euclid** dựa trên tính chất:

$$\gcd(a, b) = \gcd(b, a \bmod b)$$

Lặp lại cho đến khi $b = 0$, khi đó $\gcd(a, 0) = a$.

Chạy tay: $\gcd(48, 18)$



Tiếng Việt



$$48 \bmod 18 = 12$$

$$\rightarrow \gcd(18, 12)$$

Bước 2: $\gcd(18, 12)$

$$18 \bmod 12 = 6$$

$$\rightarrow \gcd(12, 6)$$

Bước 3: $\gcd(12, 6)$

$$12 \bmod 6 = 0$$

$$\rightarrow \gcd(6, 0)$$

Bước 4: $\gcd(6, 0)$

$$b = 0 \rightarrow \text{trả về } a = 6$$

Kết quả: $\gcd(48, 18) = 6 \checkmark$

Code

C++

C++ (iterative)

Python

```
int gcd(int a, int b) {  
    return b == 0 ? a : gcd(b, a % b);  
}
```

```
int gcd(int a, int b) {  
    while (b != 0) {  
        int temp = b;  
        b = a % b;  
        a = temp;  
    }  
    return a;  
}
```



Tiếng Việt



```
while b != 0:
    a, b = b, a % b
return a
```

Trong C++17 trở lên, có thể dùng `std::gcd` trong `#include <numeric>`.

Độ phức tạp: $O(\log(\min(a, b)))$.

2.3. Ví dụ khác: gcd(252, 105)

```
gcd(252, 105): 252 mod 105 = 42 → gcd(105, 42)
gcd(105, 42): 105 mod 42 = 21 → gcd(42, 21)
gcd(42, 21): 42 mod 21 = 0 → gcd(21, 0) = 21
```

Kết quả: $\text{gcd}(252, 105) = 21$.

2.4. LCM — Bội chung nhỏ nhất

$\text{lcm}(a, b)$ là số nguyên dương **nhỏ nhất** mà cả a và b đều là ước.

Công thức:

$$\text{lcm}(a, b) = \frac{a \times b}{\text{gcd}(a, b)}$$

Ví dụ:

$$\text{lcm}(12, 18) = \frac{12 \times 18}{6} = 36$$

Code — chú ý tránh tràn số

C++

Python

```
long long lcm(long long a, long long b) {
    return a / gcd(a, b) * b; // chia trước, nhân sau
}
```

```
def lcm(a, b):
    return a // gcd(a, b) * b
```



```
// SAI – có thể tràn
long long lcm_wrong(long long a, long long b) {
    return a * b / gcd(a, b);
}
```

3. Số nguyên tố (Prime Numbers)

3.1. Số nguyên tố là gì?

Số nguyên tố là số nguyên > 1 mà chỉ có đúng 2 ước: 1 và chính nó.

Các số nguyên tố đầu tiên: 2, 3, 5, 7, 11, 13, 17, 19, 23, 29, ...

Lưu ý:

- 1 **không** phải số nguyên tố
- 2 là số nguyên tố chẵn **duy nhất**

3.2. Kiểm tra số nguyên tố — $O(\sqrt{n})$

C++ Python

```
bool is_prime(long long n) {
    if (n < 2) return false;
    for (long long i = 2; i * i <= n; i++) {
        if (n % i == 0) return false;
    }
    return true;
}
```



Tiếng Việt



```
if n < 2:
    return False
i = 2
while i * i <= n:
    if n % i == 0:
        return False
    i += 1
return True
```

3.3. Phiên bản tối ưu hơn

C++

Python

```
bool is_prime(long long n) {
    if (n < 2) return false;
    if (n < 4) return true;      // 2, 3
    if (n % 2 == 0) return false;
    if (n % 3 == 0) return false;

    for (long long i = 5; i * i <= n; i += 6) {
        if (n % i == 0 || n % (i + 2) == 0)
            return false;
    }
    return true;
}
```



Tiếng Việt



```
if n < 2:
    return False
if n < 4:
    return True
if n % 2 == 0 or n % 3 == 0:
    return False

i = 5
while i * i <= n:
    if n % i == 0 or n % (i + 2) == 0:
        return False
    i += 6
return True
```

Giải thích: Mọi số nguyên tố lớn hơn 3 đều có dạng $6k \pm 1$.

3.4. Ví dụ kiểm tra

```
is_prime(17)?
i=2: không chia hết
i=3: không chia hết
i=4: không chia hết
i=5: 5*5 > 17 → dừng
→ 17 là số nguyên tố ✓

is_prime(15)?
i=2: không chia hết
i=3: 15 % 3 = 0
→ 15 không phải số nguyên tố ✓
```

4. Sàng Eratosthenes — Tìm tất cả số nguyên tố đến N

4.1. Ý tưởng



so nguyên tố den N trong $O(N \log \log N)$.

4.2. Thuật toán

1. Tạo mảng `is_prime[0..N]`, ban đầu gán `true`
2. Gán `is_prime[0] = is_prime[1] = false`
3. Duyệt i từ 2 đến $\sqrt{N}$
4. Nếu `is_prime[i] = true`, đánh dấu các bội của i từ i^2 là `false`

4.3. Minh họa với $N = 30$

Ban đầu, coi mọi số từ 2 đến 30 là nguyên tố.

Bước $i = 2$: loại 4, 6, 8, 10, ..., 30

Bước $i = 3$: loại thêm 9, 15, 21, 27

Bước $i = 4$: đã bị loại, bỏ qua

Bước $i = 5$: loại thêm 25

Kết quả: {2, 3, 5, 7, 11, 13, 17, 19, 23, 29}

4.4. Code

C++

Python

```
const int MAXN = 10000000;  
bool is_prime[MAXN + 1];  
  
void sieve() {  
    fill(is_prime, is_prime + MAXN + 1, true);  
    is_prime[0] = is_prime[1] = false;  
  
    for (int i = 2; 1LL * i * i <= MAXN; i++) {  
        if (is_prime[i]) {  
            for (int j = i * i; j <= MAXN; j += i) {  
                is_prime[j] = false;  
            }  
        }  
    }  
}
```



```

is_prime = [True] * (n + 1)
if n >= 0:
    is_prime[0] = False
if n >= 1:
    is_prime[1] = False

i = 2
while i * i <= n:
    if is_prime[i]:
        j = i * i
        while j <= n:
            is_prime[j] = False
            j += i
        i += 1

return is_prime

```

Tại sao bắt đầu từ i^2 ? Vì các bội nhỏ hơn đã bị loại bởi các số nhỏ hơn trước đó rồi.

5. Số học Modular (Modular Arithmetic)

5.1. Tại sao cần modular?

Trong CP, đáp án có thể rất lớn, nên đề thường yêu cầu in kết quả theo modulo $10^9 + 7$.

$$10^9 + 7 = 1000000007$$

Đây là một số nguyên tố rất hay dùng.

5.2. Các tính chất cơ bản

$$(a + b) \bmod m = ((a \bmod m) + (b \bmod m)) \bmod m$$

$$(a \times b) \bmod m = ((a \bmod m) \times (b \bmod m)) \bmod m$$

$$(a - b) \bmod m = ((a \bmod m) - (b \bmod m) + m) \bmod m$$

Lưu ý: Phép chia không áp dụng kiểu này một cách trực tiếp.

5.3. Code



```
const int MOD = 1e9 + 7;

int add(int a, int b) {
    return ((long long)a + b) % MOD;
}

int mul(int a, int b) {
    return (1LL * a * b) % MOD;
}

int sub(int a, int b) {
    return ((long long)a - b + MOD) % MOD;
}
```

```
MOD = 10**9 + 7

def add(a, b):
    return (a + b) % MOD

def mul(a, b):
    return (a * b) % MOD

def sub(a, b):
    return (a - b + MOD) % MOD
```

Lỗi phổ biến: quên ép `long long` trong C++ khi nhân hai số lớn.

6. Lũy thừa nhanh (Fast Exponentiation)

6.1. Bài toán

Tính $a^b \bmod m$ với b rất lớn, có thể tới 10^{18} .

Cách nhân a với chính nó b lần là $O(b)$, quá chậm.

6.2. Ý tưởng



$$a^b = (a^{b/2})^2$$

- Nếu b lẻ:

$$a^b = a \times a^{b-1}$$

Mỗi bước giảm số mũ đi khoảng một nửa, nên tổng thời gian là $O(\log b)$.

6.3. Chạy tay: $3^{13} \bmod 100$

Ta có:

$$13 = (1101)_2 = 8 + 4 + 1$$

Nên:

$$3^{13} = 3^8 \times 3^4 \times 3^1$$

Tính lần lượt:

```
3^1 mod 100 = 3
3^2 mod 100 = 9
3^4 mod 100 = 81
3^8 mod 100 = 61
```

Rồi nhân các phần cần thiết:

$$3 \times 81 \times 61 \bmod 100 = 23$$

Vậy:

$$3^{13} \bmod 100 = 23$$

6.4. Code

C++

Python



Tiếng Việt



```
long long res = 1;
a %= m;

while (b > 0) {
    if (b % 2 == 1) {
        res = res * a % m;
    }
    a = a * a % m;
    b /= 2;
}

return res;
}
```

```
def power(a, b, m):
    res = 1
    a %= m

    while b > 0:
        if b % 2 == 1:
            res = (res * a) % m
        a = (a * a) % m
        b //= 2

    return res
```

6.5. Ví dụ sử dụng

C++ Python

```
cout << power(2, 100, 1000000007) << endl;    // 976371285
cout << power(3, 1000000, 1000000007) << endl;
```

```
print(power(2, 100, 1000000007))    # 976371285
print(power(3, 1000000, 1000000007))
```



Tiếng Việt



7. Tổng hợp: Các lỗi phổ biến

LỖI	HẬU QUẢ	CÁCH TRÁNH
Duyệt ước đến n thay vì $\sqrt{n}$	TLE khi n lớn	Dùng <code>i * i <= n</code>
LCM: nhân trước chia sau	Tràn <code>long long</code>	<code>a / gcd(a, b) * b</code>
Quên 1 không phải nguyên tố	WA	Kiểm tra <code>n < 2</code>
Sàng: bắt đầu từ $2i$ thay vì i^2	Chậm hơn	Bắt đầu từ <code>i * i</code>
Modular: quên ép <code>long long</code> khi nhân	Tràn <code>int</code>	Dùng <code>1LL * a * b</code>
Modular: quên cộng MOD khi trừ	Kết quả âm	<code>(a - b + MOD) % MOD</code>
Lũy thừa: dùng vòng lặp $O(b)$	TLE	Dùng binary exponentiation

Tóm tắt bài học

KHÁI NIỆM	KỸ THUẬT CHÍNH	ĐỘ PHỨC TẠP
Liệt kê ước số	Duyệt đến $\sqrt{n}$, thêm cặp	$O(\sqrt{n})$
GCD	Thuật toán Euclid	$O(\log(\min(a, b)))$
LCM	$a / \gcd(a, b) \times b$	$O(\log(\min(a, b)))$
Kiểm tra nguyên tố	Duyệt đến $\sqrt{n}$	$O(\sqrt{n})$
Sàng nguyên tố	Sàng Eratosthenes	$O(N \log \log N)$
Modular arithmetic	Mod sau mỗi phép tính	$O(1)$ mỗi phép
Lũy thừa nhanh	Bình phương liên tiếp	$O(\log b)$

Bây giờ hãy thử sức với các bài tập. Bắt đầu từ các bài về ước số và GCD, rồi tiến đến bài modular và lũy thừa nhanh nhé!

Bài tập



Tiếng Việt



Tìm UCLN, BCNN	0 / 10
Số nguyên tố	0 / 10
Ước số chung bản nâng cao	0 / 15
Ước số chung lớn nhất (Khó)	0 / 15
CSES - Trailing Zeros Số không ở cuối	0 / 15
CSES - Bit Strings Xâu nhị phân	0 / 15
Số lượng ước số của n	0 / 20
KT Số nguyên tố	0 / 20
CSES - Exponentiation Lũy thừa	0 / 20
CSES - Two Knights Hai quân mã	0 / 20
CSES - Counting Divisor Đếm ước	0 / 25

proudly powered by **DMOJ** | [developed by LQDJudge team](#) Tiếng Việt (vi)